Reading Files

Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io



A program's variables disappear when it stops. A **file** keeps data on disk, so it can be read again later.

plain text files	**other files**
`.txt` `.csv` `.py`	`.pdf` `.docx` `.png`
only characters and newlines; any text editor can open and show them	data in a special format; they need the right program to open them, and are often much bigger

In this course we read plain text files.

Before reading a file, a program must **open** it. A `with` statement opens the file and closes it again when its block ends.

file name
a string

file object
used to read the file

indented block
all work with the file

```
with open('movies.txt') as movie_file:
    # read from movie_file here
    # (the file is open)
# movie_file is closed here
```

open looks for the file in the same folder as the program. If it isn't there, open raises a `FileNotFoundError`.

A `for` loop over a file object goes through the file one **line** at a time. Each line is a string that ends with the newline character `'\n'`.

movies.txt

```
Whiplash
Shrek
Coraline
```

```
lines = []
with open('movies.txt') as movie_file:
    for line in movie_file:
        lines.append(line)
```

After the loop, `lines` refers to `['Whiplash\n', 'Shrek\n', 'Coraline\n']`. To get a line without its newline, use `line.strip()`.

A file object remembers how far it has read. Once a loop has gone through the file, a second loop has nothing left to read.

```
>>> with open('movies.txt') as movie_file:
...     for line in movie_file:
...         print(line.strip())
...     for line in movie_file:
...         print('again', line.strip())
...
Whiplash
Shrek
Coraline
```

the second loop prints nothing

Reading a line **consumes** it. To start over, open the file again.

.csv files

A `.csv` (comma-separated values) file stores a table as plain text: one row per line, with commas between the entries in a row.

watchlist.csv

```
Title,Year,Genre,Rating
Whiplash,2014,Drama,9
Shrek,2001,Comedy,8
Coraline,2009,Animation,8
```

Title	Year	Genre	Rating
Whiplash	2014	Drama	9
Shrek	2001	Comedy	8
Coraline	2009	Animation	8

The first line is often a **header** that names the columns. Spreadsheet programs can open and save `.csv` files.

Reading a .csv file line by line gives each row as one string. It would be more useful to have each row as a `list` of its entries.

```
>>> rows = []
>>> with open('watchlist.csv') as watch_file:
...     for line in watch_file:
...         rows.append(line)
...
>>> rows[1]
'Whiplash,2014,Drama,9\n'
```

↓ what we would like

```
['Whiplash', '2014', 'Drama', '9']
```

The csv module can do this for us.

A **module** is a file of Python code that other programs can use.
To load a module, we use an **import** statement. We can then use its functions and variables with dot notation.

import a module

```
>>> import math
>>> math.sqrt(16)
4.0
>>> math.pi
3.141592653589793
```

import one name from it

```
>>> from math import sqrt
>>> sqrt(25)
5.0
```

Put imports at the top of a file. Python comes with many modules, such as csv for reading .csv files.

A **csv reader** goes through an open file and gives back each row as a `list` of strings.

```
>>> import csv
>>> with open('watchlist.csv') as watch_file:
...     reader = csv.reader(watch_file)
...     for row in reader:
...         print(row)
...
['Title', 'Year', 'Genre', 'Rating']
['Whiplash', '2014', 'Drama', '9']
['Shrek', '2001', 'Comedy', '8']
['Coraline', '2009', 'Animation', '8']
```

Every item is a string, even `'2014'`. Use the reader inside the `with` block, while the file is open.

`next(reader)` reads one row and returns it. Like a `for` loop, it consumes that row, so a loop afterwards starts at the next row. This is how to skip a header.

```
>>> with open('watchlist.csv') as watch_file:
...     reader = csv.reader(watch_file)
...     header = next(reader)
...     for row in reader:
...         print(row[0], 2026 - int(row[1]))
...
Whiplash 12
Shrek 25
Coraline 17
```

`header` refers to `['Title', 'Year', 'Genre', 'Rating']`, in case the program needs the column names.

A function that reads a file usually takes the **open file** as a parameter. Its type is `TextIO`, from the `typing` module.

```
from typing import TextIO

def count_lines(data_file: TextIO) -> int:
    """Return the number of lines in data_file."""
    total = 0
    for line in data_file:
        total = total + 1
    return total
```

The caller must open and close the file:

```
with open('movies.txt') as movie_file:
    num_movies = count_lines(movie_file)
```

The function only reads, so it works for any open file in the right format.